

Cache Hierarchy Design Under Hard Caps with a Council of Agents: A CHIA Loop That Measures and Gates Itself

First Author, Second Author

Affiliation

first.author@example.org

Abstract—Choosing a cache hierarchy means setting the sets, ways, prefetcher, replacement policy and queue depth of every level at once, under a silicon budget and a power budget, with one simulation per design to judge it. We search 13 such knobs of a three-level hierarchy, 6.6 million designs, under caps of 4.0 mm² and 1.0 W, with a council of Gemini agents built as a CHIA loop: an analyst that diagnoses the current design, four specialists that each propose one move, a statistician with a Gaussian process, and a repair that shrinks a design the power cap refused. The council reaches the quality random search has after 1000 designs in 39 to 44 designs. CHIA gave the loop its nodes, its cluster and its profiler; we added what the loop needed and CHIA did not have: a Gemini node that bills thinking tokens, a ChampSim node that builds a whole hierarchy, an evaluation ledger, a spend view over the profiler log, and two gates in which a cheap calibrated decider says which agent calls and which briefing sections are worth paying for. Applied, the gates reach the same final IPC for 22 to 30% less model spend. The blocks travel: on CHIA’s own CIRCT loop the call gate cuts the assess stage from 16 to 13 agent turns with the same issues sent forward. Everything reproduces from the repository with one command and no key.

I. INTRODUCTION

A cache hierarchy is usually set by sweeps and experience: one level at a time, one knob at a time, the rest held. The knobs do not behave that way. A larger last level costs power a smaller second level could have saved, a prefetcher that helps one level floods the next, and a cap on silicon or power turns every gain into a trade. Each design costs a simulation, so the currency of the search is the number of designs measured.

CHIA [1] is a framework for building loops in which agents propose and tools measure, every step a task on a cluster with a profiler recording it. We built such a loop for this problem: a council of agents that reads the simulator’s counters, diagnoses where the time and the power go, proposes designs and has them measured five at a time. Against random search over the same feasible space it reaches the quality random has after 1000 designs in 39 to 44 designs, under the caps and without them.

That result is what such loops are usually judged by, and it says nothing about what the loop cost or where. The CHIA paper [1] names the gap: “data collection and profiling of both results and of the loop itself built in as first-class citizens”. We took that literally. The loop’s model calls, builds, simulations, candidates and gate decisions are all CHIA tasks, so one profiler log per run holds all of them, and the blocks we added

read that log or add events to it: a spend view that sums it per node and model, a ledger that scores every candidate the way search methods are compared, and two gates in which a cheap calibrated decider answers one yes-or-no question before an optional agent call or per section of a briefing. Run in shadow the gates only record, which told us that 57% of the specialists’ calls ended in a hold and which ones a decider could see coming; applied, they gave the same final IPC for 22 to 30% less model spend.

The paper takes the story in that order: the problem (Section II), the council as a CHIA loop (Section III), the result against random (Section IV), the cost and the gates (Section V), the blocks on CHIA’s own CIRCT loop (Section VI), what we did not expect (Section VII), and the artifact (Section VIII).

II. THE DESIGN PROBLEM

The chip is ChampSim’s [2] stock core widened to 6-wide issue with a 384-entry reorder buffer at 3.8 GHz in 22 nm, with DDR5-4800 over two channels. The search moves 13 knobs: sets and ways of the L1D, L2 and LLC on per-level ladders, the prefetcher of each level, the replacement policy of the L2 and LLC, and the miss-handling depth of both, 6.6 million designs in all. Two caps make it a design problem rather than a sweep: 4.0 mm² of cache silicon and 1.0 W of cache plus off-chip power, and a design may not run the workload slower than the stock chip does. The two caps are known at different times, and that shapes the loop. Silicon and leakage come from CACTI 7 [3] at 22 nm and are known before a design is simulated, so a design over the area cap is never run. Power is CACTI’s energies times the simulator’s hit and miss counters plus 15 pJ per bit moved off chip, so it is known only after. A design over the power cap is measured and read by the agents, and never becomes the incumbent. The stock chip itself draws 1.095 W on the workload and breaks the cap, so the reference is the stock with its LLC halved: 1.96 mm², 0.805 W, IPC 0.935. The caps bite: 36% of the council’s candidates were refused after measurement, 21% by power and 19% by the speed floor, some by both. The area cap rarely binds, since good designs are small: the best draws 0.67 W on 1.33 mm². A second regime drops both caps.

The workload is the llama2 inference trace of the ChampSim ML suite, simulated at 1M warm-up and 2M instructions;

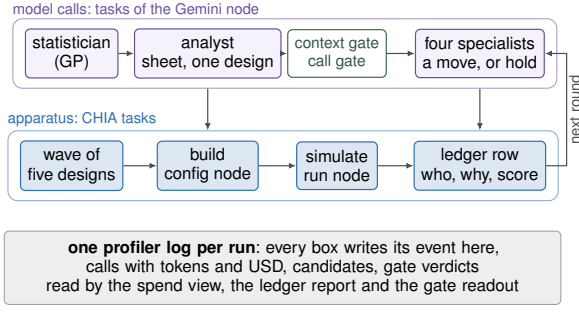


Fig. 1. One round of the council. Violet: model calls; blue: apparatus; green: the decider’s two gates. Everything lands in one profiler log per run.

40 designs re-measured at 5M/25M rank the same (Spearman 0.986, top-5 overlap 80%). We picked the hardest of six ML traces on purpose, the one with the most headroom over the stock and the deepest coupling between knobs, so the council is tested where a sweep fails.

III. THE COUNCIL AS A CHIA LOOP

One round. Fig. 1 is the loop. Round one measures the analyst’s opening design and 19 random feasible designs. Every later round measures one wave of five: a statistician, a Gaussian process over every design this run measured, proposes its top pick; the analyst writes a sheet (a verdict per level with its evidence, the knobs that failed alone, the cap headroom) and one whole design; four specialists, for prefetching, geometry, replacement and concurrency, each propose one move on their own knobs or hold. The wave takes at most five, in this order: the statistician’s pick, the analyst’s design, a gain the power cap refused last round shrunk to fit, the specialists’ moves, and untried single-knob moves at the level the sheet blames to fill what is left. The incumbent is the best feasible design measured, whoever proposed it. A move already measured against it is a hold. One model serves every call, Gemini 3.1 Pro on Vertex at temperature 0.7 with a thinking budget of 4096 tokens.

What CHIA gave and what we added. Every box in Fig. 1 is a CHIA task on a Ray [4] cluster; each run connects under its own name with its own profiler collector, so a run’s log is a complete record of it, and simulations run through CHIA’s own ChampSim run node. Three nodes did not exist and we wrote them in CHIA’s layout. CHIA’s Vertex node drives a tool loop under a fixed generation config and leaves the thinking tokens out of its usage record; our Gemini node makes one JSON call at a set temperature and thinking budget and records every token, the USD and the caller. CHIA’s build node writes one prefetcher at one level; ours builds from a whole configuration. CHIA’s run node keeps a subset of the simulator’s record that the analyst reads past, so a 7-line patch returns the whole of it and the loop’s parse gives identical numbers. Every design measured is one ledger row and one `candidate` event in the log.

Protocol. The budget unit is a design: wall time at this fidelity shows no advantage either way, since the model’s minute a round offsets fewer rounds. Random search draws

TABLE I
THE COUNCIL AGAINST RANDOM SEARCH, MEANS OVER SEEDS. DESIGNS TO TARGET COUNTS A SEED AT THE BUDGET OF 60

	caps on	caps off
target (random’s mean final at 1000 designs)	1.4329	1.5001
council designs to the target	44	39
fewer simulations than random	23×	25×
new bests: the model’s designs and repairs	62%	40%
new bests: statistician / single-knob fills	25% / 13%	36% / 24%

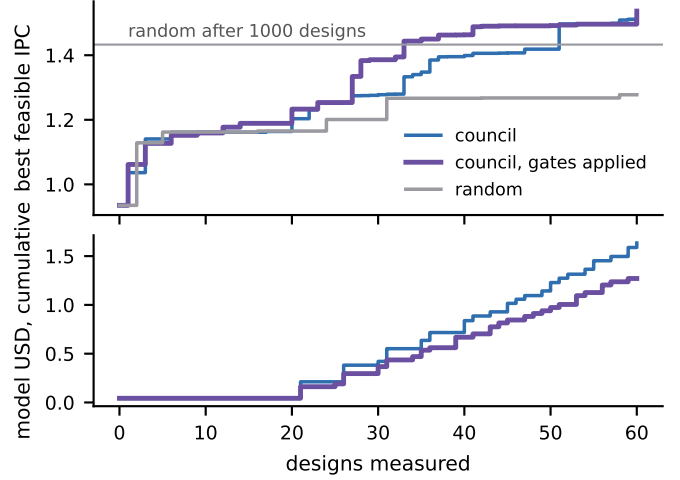


Fig. 2. Caps on, means over seeds: the best feasible IPC as designs are measured, and the model spend of the two council arms beneath it. Random’s line is its 1000-design mean.

1000 feasible designs, three seeds; the council runs 60 designs or 20 without a new incumbent, four seeds per arm and regime. The target is the mean of random’s best after 1000 designs over its seeds; a seed that spends its budget under the target counts at the budget, 60 designs, with its final IPC beside it. Every arm is scored from the same ledger rows, random’s rebuilt from its seed and the result table.

IV. AGAINST RANDOM SEARCH

The result. Random search (Table I, Fig. 2) climbs early, because a fifth of the feasible space beats the halved stock, and then stalls. The council passes random’s 1000-design quality at 44 designs under the caps and at 39 without. Fifteen of its sixteen runs end above that quality; the other ends at 1.497 against 1.5001.

Who found the improvements. The ledger records who proposed every new best design, and the answer differs between the two regimes. Under the caps the model did most of the work: its whole designs and its repairs of refused gains produced 62% of the new bests, the statistician 25%, single-knob fills 13%. Without the caps the statistician led with 36%, single-knob fills took 24%, and the model 40%.

Why. A cap makes the objective jump. A design can be faster and still be refused, and a surrogate fitted to IPC does not see that wall; a model that reads the counters does, and can say which level to shrink. Without the caps the landscape is smooth, and the surrogate’s interpolation is the better guesser.

The caps decide who wins, not whether the search wins, and a ledger that records who won tells the two apart per problem.

Against Bayesian optimisation. We also ran a standard Bayesian optimisation, a random-forest surrogate with expected improvement and the same waves of five, two seeds per regime. It needs as many designs as the council to reach random’s 1000-design quality. Without the caps it ends 1% above the council’s final IPC; under the caps it ends 2.5% below, and half to two thirds of its designs came back refused by the power cap, against a third of the council’s. That is the pattern above once more: a surrogate guesses well on a smooth landscape, and where the cap turns a faster design into a refused one, an agent that reads the counters does better.

V. CHEAPER AT THE SAME IPC

What the instruments told us. A run’s model bill is 1.6 to 1.7 USD; the share saved is what scales, with more seeds, full fidelity, bigger loops, and every loop in which most optional calls end in a hold. The recipe above is the sixth version of the council, and the ledger chose most of what is in it. Its rows showed that four rules of earlier versions never paid and that one setting stalled half the seeds, and the recipe changed accordingly. The per-call rows, priced by the spend block, showed what the bill was made of: input 49%, thinking 37%, answers 14%, and the analyst 44% of it. The specialists’ rows showed that 57% of their calls ended in a hold. The specialist read its briefing and had nothing to add. Two levers were left: the calls that end in a hold, and the briefing every call carries.

The gate. A gate in front of a call needs a verdict that is cheap, fast and calibrated, not a second agent. TypeSafe’s Jev [5] answers a typed yes-or-no question with a probability, at 0.042 USD per million input tokens; a round of four questions costs about a tenth of a cent and returns in a second. The call gate asks, per specialist and round, whether the analyst’s sheet gives evidence for moving that concern’s knobs; Listing 1 is the whole of it. The context gate asks, per section of a briefing, whether the task needs it, from the caller’s one-line description of the section, and rates the task’s difficulty, which sets the thinking budget. Both have two modes. In shadow every call is still made and the verdict is recorded beside what the call then produced; in apply the calls under the threshold are not made and the sections rated unneeded are dropped. That pair is the value: shadow prices the gate, apply collects it, and nothing is skipped on faith.

What the shadow showed, and what applying it saved.

Table II: over the 304 specialist calls of the eight shadow runs, a threshold of 0.4 would have skipped 40%, 92 holds and 29 proposals, of which two gained more than 0.005 IPC; of the 20 gains above 0.005 among all 304 calls, 18 carried a verdict of 0.4 or more. The context gate drops 16 to 17% of the briefing tokens once shown a description per section; shown the sections’ first characters instead, it had rated the evidence unneeded, a negative we kept. Applied together, with a specialist’s thinking capped when its task is not rated hard, the council consulted 2.4 to 2.8 specialists a round instead of four and reached the same final IPC for 22% less model

```
gate = CallGate(JevDecider(), threshold=0.4, mode="apply")
state = {"sheet": sheet_text(sheet)[:2500],
        "current_design": knobs_text(incumbent),
        "statistician_expects": expects[:800]}
questions = {}
for name in CONCERNS: # prefetch, geometry, replacement, ...
    questions[name] = {
        "instructions": f"Does the sheet give evidence "
                        f"for moving the {name} knobs?",
        "criteria": {"true": "its findings point here",
                    "false": "nothing does; a hold is free"}}
verdicts = gate_and_log(gate, state, questions, label="round")
for name in verdicts: # shadow: every call made
    if verdicts[name]["consult"]:
        ask_specialist(name) # apply: the rest not called
```

Listing 1. The call gate in front of the council’s specialists. The state and the questions are the loop’s, the rest is the block’s; ask_specialist is one task of the Gemini node.

TABLE II

THE CALL GATE IN SHADOW OVER 304 SPECIALIST CALLS OF EIGHT RUNS (GAINS LOST: SKIPPED PROPOSALS THAT GAINED MORE THAN 0.005 IPC), THEN BOTH GATES APPLIED, 4 SEEDS PER ARM AND REGIME

Shadow: what the gate would have skipped	threshold	skipped	holds	proposals	gains lost
	0.3	93 (31%)	68	25	2
	0.4	121 (40%)	92	29	2
	0.5	147 (48%)	112	35	3
Applied at 0.4, with the context gate; “all calls” is the ungated baseline					
		caps on		caps off	
		all calls	gated	all calls	gated
model USD per run		1.63	1.27	1.71	1.20
model calls per run		47.2	34.0	49.8	31.0
final IPC, mean of seeds		1.512	1.537	1.521	1.513
less model spend		22%		30%	

spend under the caps and 30% without (Fig. 2), the decider’s cost included. Nearly all of the saving is the calls not made, 94 to 98% of it. The trimmed briefings cut a specialist call’s input by 7 to 12%, but the cost of a call made was flat, since thinking and answer lengths vary by more than that.

VI. THE BLOCKS TRAVEL

Everything the loop needed is a module and a test in CHIA’s tree (Table III), installed into the released package by one script; the tests pass inside it, and the CLI’s new format sums a log CHIA’s own Vertex node wrote.

On CHIA’s own loop. CHIA’s CIRCT issue solver begins with an *assess* turn in which an agent reads a GitHub issue and decides whether it is a bug with one clear fix; an issue that is not a bug is set aside after that turn, so it costs the turn. We re-ran the turn under the blocks: the paper’s own prompt, the 16 issues of its Table 5, CHIA’s Vertex node under CHIA’s profiler, and the call gate asking the decider “does this issue report a defect?” before the turn, at 0.65. Table IV gives means over passes, since answers vary. The gate skipped the same three of the five non-bugs in both gated passes and none of the eleven bugs, so the paper’s seven issues proceed to reproduction as before; turns and tokens fell 19%, the decider’s 16 decisions cost 0.0008 USD. The saving is bounded by the share of non-bugs in the stream, five of sixteen here; what the gate buys is that the expensive turn is spent only where it is needed, with no bug lost.

Five findings about CHIA’s nodes, filed upstream as fixes. A prompt that mentions an undeclared tool makes the Vertex

TABLE III
THE BLOCKS, WHAT EACH READS OR RECORDS, AND WHERE EACH WAS
PROVEN. MODULES UNDER CHIA/TRACE, CHIA/MODELS AND
CHIA/SIMULATORS

Block	What it reads or records	Proven on
Spend view and cap	USD and tokens by kind per node and model, from the events. <code>chia viz-profile -format spend</code> ; <code>LLMSpend</code> stops a loop before the call that would pass a cap.	CHIA's Vertex node under a local Ray; the CIRCT stage; the council's own log
Evaluation ledger	One row per candidate: who, why, what it measured, which caps it broke. From the rows: best-so-far, evaluations to a target, plateau, speedups, rank agreement.	Every arm of Sec. IV
Decider and call gate	One yes-or-no question per optional call, answered by Jev from the caller's state; shadow records, apply skips.	The council (Sec. V); CIRCT: 16 to 13 turns
Context gate	Per section of a briefing, the decider judges the caller's one-line description and rates the task's difficulty, which sets the thinking budget.	The council: 16–17% fewer briefing tokens
Gemini JSON node	One JSON answer at a set temperature and thinking budget; the event carries every token kind, the USD and the caller.	Every model call of the council
ChampSim configuration build	Builds ChampSim from a whole configuration and returns CHIA's own build result. A 7-line patch returns the simulator's raw record.	Every build and run of the council; IPC equal to the loop's own build

TABLE IV
CHIA'S ASSESS STAGE OVER THE PAPER'S 16 ISSUES (11 BUGS, 5 NOT),
UNGATED AND GATED. AGREE: ISSUES ON WHICH THE VERDICT
MATCHED THE PAPER'S, OF 16. TOKENS AND USD ARE THE SPEND VIEW
OVER EACH PASS'S PROFILER LOG

	turns	agree	skipped		tokens	USD
			bugs	non-bugs		
ungated, 3 passes	16	14.3	0 of 11	0 of 5	37.7k	0.095
gated at 0.65, 2 passes	13	14.5	0 of 11	3 of 5	30.4k	0.081
change	—19%				—19%	—15%

node return success with an empty answer (14 of 16 issues came back blank before the prompt was changed); a rate limit raises at once, so a batch dies on the first 429; its usage record never reads the thinking tokens; the ChampSim run node keeps the first DRAM channel only and no merged misses; and both build nodes call `nproc`, which a Mac does not have.

VII. WHAT THE INSTRUMENTS FOUND

The model alone does not search. Without the statistician, the repairs and the fills, the specialists ran out of proposals at 25 to 28 designs and the runs ended at IPC 1.15 to 1.17 in both regimes. **Grounding is a gauge, not a filter.** Asked whether the sheet supports each of 377 proposals, the decider's verdict did not predict which gained, so it measures prompt discipline and must not skip anything. **The decider cannot judge “one clear fix”** (10 of 11 CIRCT bugs rated unclear), which is why the gate asks the narrower question.

VIII. ARTIFACT, RELATED WORK AND LIMITATIONS

Artifact. `python3 council_loop/demo.py` replays every table of this paper from the shipped records in under a second with no key, no cluster and no simulator; `check.sh` installs the released CHIA into a clean environment, copies the blocks in, applies the two patches and runs the 33

tests inside the package. The fixes go upstream; the blocks stay in this repository in CHIA's layout. The release tag `frozen-council` is the version the runs used. All of it is at https://github.com/mateobuscarons/chia_hackathon.

Related work. AgentDSE [6] and the CHIA studies [1] search architectural design spaces with reasoning agents and report the outcome. The ledger's metrics are the ones predictive design-space exploration [7] and Bayesian optimisation [8] compare methods with, evaluations to a target and best-so-far per evaluation, here scoring an agentic loop from its own records. FrugalGPT [9] lets a cheap model decide whether an expensive one is called; the call gate is that decision made by a calibrated decider [5] at the loop level, with a shadow mode that prices it before it is trusted.

Limitations. Four seeds per arm and regime bound the quality claim; the shadow counterfactual, 304 calls, is the larger sample. One trace at 1M/2M instructions carries the search, with a 40-design check at 5M/25M. The caps are ours, taken from the stock chip's own draw rather than from a product. Against the strongest random seed instead of the mean, the council needs 44 and 46 designs.

ACKNOWLEDGEMENTS

We thank the CHIA developers and the A³ hackathon organisers. Claude (Anthropic) assisted with the code and the drafting.

REFERENCES

- [1] A. Cui, F. Hermida-Rivera, J. Toubes, R. Gupta *et al.*, “CHIA: An open-source framework for principled, agentic AI-driven hardware/software co-design research,” *arXiv preprint arXiv:2606.27350*, 2026.
- [2] N. Gober, G. Chacon, L. Wang, P. V. Gratz, D. A. Jiménez, E. Teran, S. Pugsley, and J. Kim, “The Championship Simulator: Architectural simulation for education and competition,” *arXiv preprint arXiv:2210.14324*, 2022.
- [3] R. Balasubramonian, A. B. Kahng, N. Muralimanohar, A. Shafiee, and V. Srinivas, “CACTI 7: New tools for interconnect exploration in innovative off-chip memories,” *ACM Trans. Architecture and Code Optimization*, vol. 14, no. 2, pp. 14:1–14:25, 2017.
- [4] P. Moritz, R. Nishihara, S. Wang *et al.*, “Ray: A distributed framework for emerging AI applications,” in *Proc. 13th USENIX Symp. on Operating Systems Design and Implementation (OSDI)*, 2018, pp. 561–577.
- [5] TypeSafe AI, “System one documentation: the noul primitive, confidence and models,” <https://docs.typesafe.ai>, 2026.
- [6] C. Wang, J. C. Shi, D. Kong, D. S. Boning, Z. Wan, Y. Du, and V. J. Reddi, “AgentDSE: Reasoning-augmented architectural design space exploration,” *arXiv preprint arXiv:2606.21836*, 2026.
- [7] E. İpek, S. A. McKee, R. Caruana, B. R. de Supinski, and M. Schulz, “Efficiently exploring architectural design spaces via predictive modeling,” in *Proc. 12th Int. Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2006, pp. 195–206.
- [8] J. Snoek, H. Larochelle, and R. P. Adams, “Practical Bayesian optimization of machine learning algorithms,” in *Advances in Neural Information Processing Systems 25 (NIPS)*, 2012, pp. 2951–2959.
- [9] L. Chen, M. Zaharia, and J. Zou, “FrugalGPT: How to use large language models while reducing cost and improving performance,” *arXiv preprint arXiv:2305.05176*, 2023.